

INGENIERÍA DE SOFTWARE II

Paralelo 102

Integrantes:

Jair Palaguachi

Dhamar Patiño

Christopher Villón

Derian Baque

Gustavo Guzmán

Workshop

Continuous Integration

PAO: 2026 - I

1. Introduction

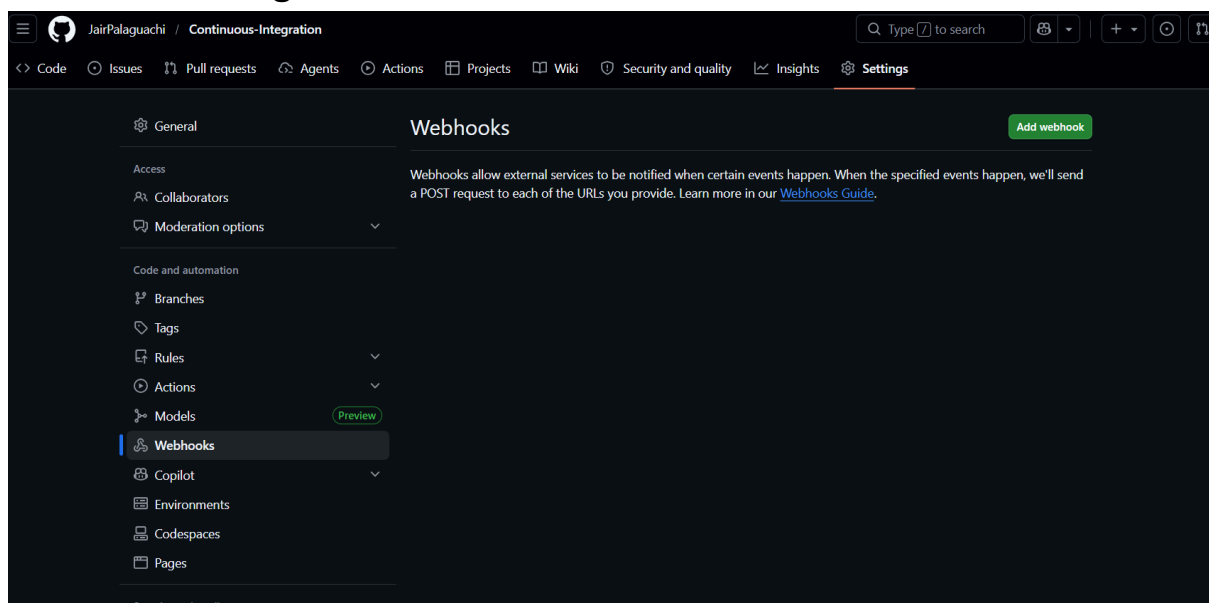
Continuous Integration (CI) is a software development practice that allows code changes to be integrated frequently into a shared repository, automatically running tests and validations to detect errors at an early stage. This methodology helps improve software quality, reduce the time required to fix defects, and facilitate collaboration among development team members.

In this workshop, a Gym Membership Management System was developed using Python, and both Jenkins and GitHub Actions were configured to automate verification processes. Jenkins was used to automatically execute unit tests whenever updates were pushed to the repository, while GitHub Actions performed code quality analysis through linting. This practice demonstrated the importance of automation in modern software development and highlighted the benefits of Continuous Integration in ensuring more reliable and maintainable software.

2. Development

Github repository: <https://github.com/JairPalaguachi/Continuous-Integration.git>

Webhook configuration:



JairPalaguachi / Continuous-Integration

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *
https://tabby-extended-strobe.ngrok-free.dev/login?from=%2Fgithub-webhook/

Content type *
application/json

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.
☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?
☒ Just the push event.
☐ Send me everything.
☐ Let me select individual events.

☒ Active
We will deliver event details when this hook is triggered.

Add webhook

Ngrok functionality 200 OK:

```
Session Status      online
Account             JairPalaguachi (Plan: Free)
Version             3.39.1-msix-stable
Region              South America (sa)
Latency             406ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://tabby-extended-strobe.ngrok-free.dev -> http://localhost:8080

Connections         ttl      opn      rt1      rt5      p50      p90
                   24       1        0.00     0.01     31.30    54.74

HTTP Requests
-----
14:34:54.978 -54 POST /github-webhook/      200 OK
```

Jenkins Configuration:

Jenkins / Todo / Nuevo Tarea

Nuevo Tarea

Enter an item name

Continuous Integration

Select an item type

Pipeline

Crear un proyecto de estilo libre

Crear un proyecto multi-configuración

Folder

Multibranch Pipeline

Organization Folder

OK

 Jenkins

Continuous Integration

Configure

Configure

General

Configurar el origen del código fuente

Triggers

Environment

Build Steps

Acciones para ejecutar después.

Enabled

General

Descripción

Plain text

☐ Desactivar ejecuciones antiguas

☐ Esta ejecución debe parametrizarse

☒ GitHub project

Project url

https://github.com/jairPalaguachi/Continuous-Integration.git

Avanzado

☐ Throttle builds

☐ Lanzar ejecuciones concurrentes en caso de ser necesario

Avanzado

Configurar el origen del código fuente

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ Ninguno

☒ Git

Repositories

Repository URL

https://github.com/jairPalaguachi/Continuous-Integration.git

Credentials

- none -

Avanzado

+ Add Repository

Branches to build

Save

Apply

 Jenkins

Continuous Integration

Configure

Configure

General

Configurar el origen del código fuente

Triggers

Environment

Build Steps

Acciones para ejecutar después.

Lanzar ejecuciones remotas (ejem: desde 'scripts')

Construir tras otros proyectos

Consultar repositorio (SCM)

Ejecutar periódicamente

☒ GitHub hook trigger for GITSCm polling

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s)

☐ Abortar la ejecución si se atasca

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☐ With Ant

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Ejecutar un comando de Windows

Comando

Visualiza la lista de variables de entorno disponibles

python -m venv env
call env\Scripts\activate
pip install pytest
pytest --junitxml=test-results.xml
deactivate

Avanzado

+ Añadir un nuevo paso

Acciones para ejecutar después.

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Publicar los resultados de tests JUnit

Ficheros XML con los informes de tests

El atributo @includes de la etiqueta <fileset> especifica dónde están los ficheros XML generados, por ejemplo: 'myproject/target/test-reports/*.xml'. El directorio base para la etiqueta <fileset> es el directorio raíz del proyecto

test-results.xml

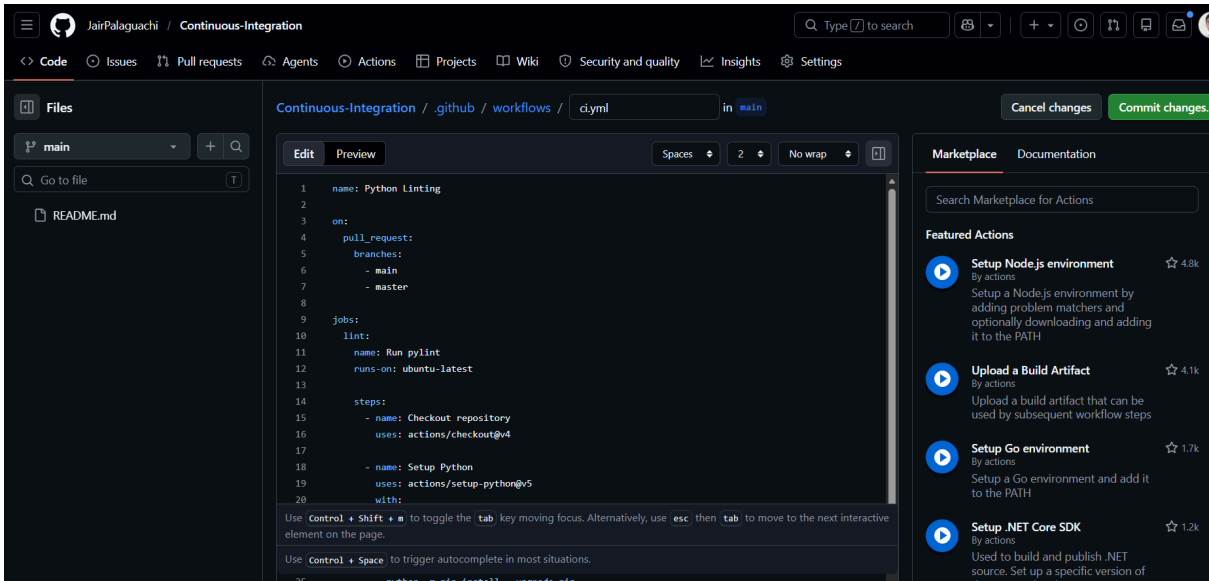
Test output retention

...

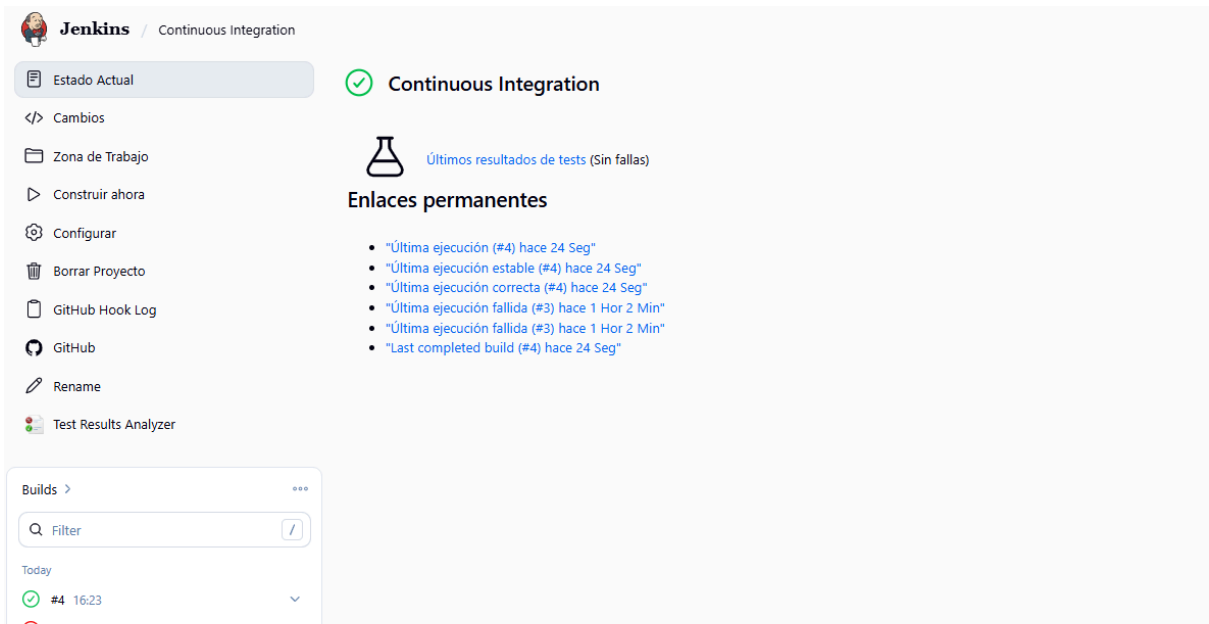
Save

Apply

Github Actions Configuration:



Final Jenkins Functionality:



Final Github Actions Configuration Functionality:

⌵

JairPalaguachi / Continuous-Integration

🔍 Type to search

🔖

+

🕒

🔍

📧

👤

<> Code

🕒 Issues

🔗 Pull requests

👤 Agents

🕒 **Actions**

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

⚙ Settings

← Python Linting

🟢 **Final version #6**

Re-run all jobs

⋮

🏠 Summary

All jobs

🟢 Run pylint

Run details

🕒 Usage

📄 Workflow file

Triggered via pull request 1 minute ago

Status

Total duration

Artifacts

👤 JairPalaguachi opened #3 BranchJP

Success

17s

–

ci.yml

on: pull_request

🟢 Run pylint

13s

🔍

–

+

Annotations

1 warning

⚠ Run pylint

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5. A...

Show more

⌵

JairPalaguachi / Continuous-Integration

🔍 Type to search

🔖

+

🕒

🔍

📧

👤

<> Code

🕒 Issues

🔗 Pull requests

👤 Agents

🕒 **Actions**

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

⚙ Settings

← Back to pull request #3

🟢 **Final version #6**

Re-run all jobs

⋮

🏠 Summary

All jobs

🟢 Run pylint

Run details

🕒 Usage

📄 Workflow file

> Annotations

1 warning

Run pylint

succeeded 1 minute ago in 13s

🔍 Search logs

🔄

⚙

> 🟢 Set up job

1s

> 🟢 Checkout repository

1s

> 🟢 Setup Python

0s

> 🟢 Install dependencies

4s

> 🟢 Run pylint

3s

> 🟢 Post Setup Python

0s

> 🟢 Post Checkout repository

0s

> 🟢 Complete job

0s

Contributions:

main.py (Derian Baque)

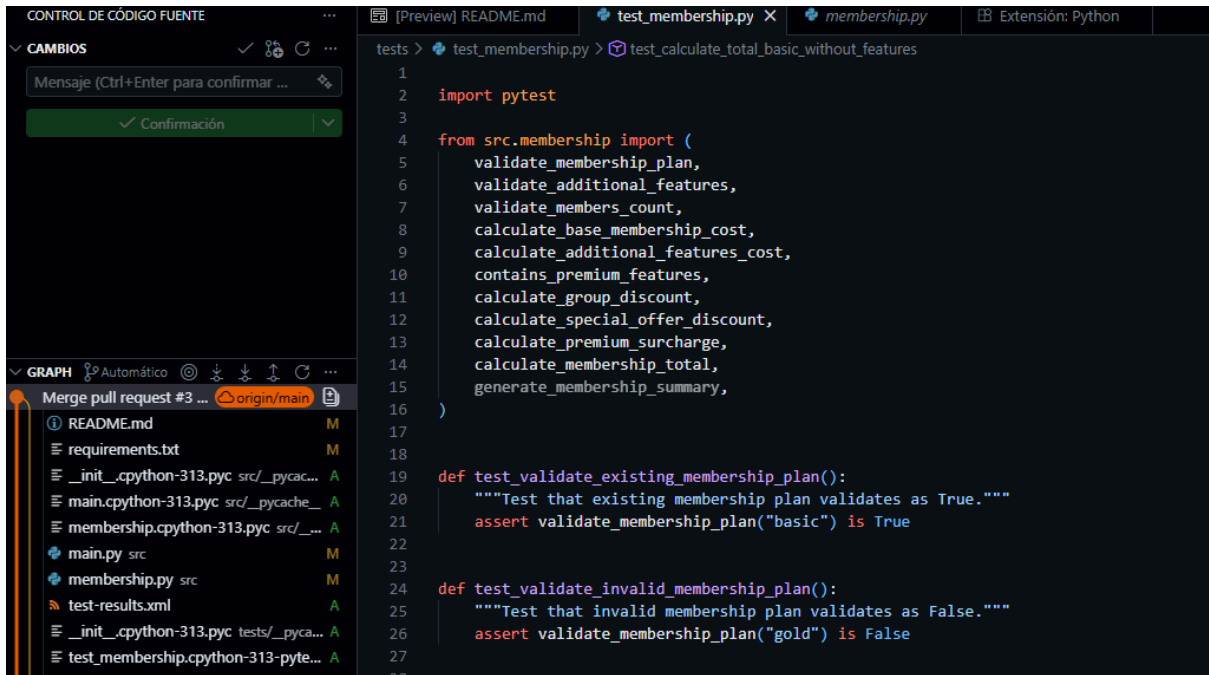
```
1  """Command-line interface for the Gym Membership Management System."""
2
3  from src.membership import (
4      get_available_plans,
5      get_available_features,
6      generate_membership_summary,
7      calculate_membership_total,
8  )
9
10
11  def display_membership_plans():
12      """Display the available membership plans."""
13      print("Available Membership Plans")
14      print("-----")
15
16      plans = get_available_plans()
17
18      for key, plan in plans.items():
19          print(f"{key}: {plan['name']} - ${plan['cost']}")
20          print("Benefits:")
21
22          for benefit in plan['benefits']:
23              print(f"    - {benefit}")
24
25          print()
26
27
28  def display_additional_features():
29      """Display the available additional features."""
30      print("Available Additional Features")
31      print("-----")
32
33      features = get_available_features()
34
35      for key, feature in features.items():
36          feature_type = "Premium Feature" if feature['premium'] else "Standard Feature"
37          print(f"{key}: {feature['name']} - ${feature['cost']} ({feature_type})")
38
39      print()
40
41
42  def get_user_features():
43      """Read additional features entered by the user."""
44      features_input = input(
45          "Enter additional features separated by commas, or press Enter for none: "
```

Jenkins and Github actions workflows configuration (Jair Palaguachi)

```
EXPLORER
CONTINUOUS-INTEGRATION
  .github\workflows
  ci.yml
  .pytest_cache
  > src
    _pycache_
    __init__.cpython-313.pyc
    main.cpython-313.pyc
    membership.cpython-313.pyc
    __init__.py
    main.py
    membership.py 1
  tests
    _pycache_
    __init__.cpython-313.pyc
    test_membership.cpython-313-pytest-9.0.3.pyc
    __init__.py
    test_membership.py
  .gitignore
  README.md
  requirements.txt

1  # Gym Membership Management System
2
3  Command-line application for managing gym memberships, calculating membership costs, applying discounts, val
4  selections, and confirming or canceling membership plans.
5
6  This project was developed as part of a Continuous Integration workshop using **Jenkins**, **Ngrok**, **Gith
7  **pytest**, and **pylint**.
8
9  ## Project Description
10
11  The Gym Membership Management System allows users to:
12
13  * View available gym membership plans.
14  * Select a membership plan.
15  * Add optional features to the selected membership.
16  * Calculate the total membership cost.
17  * Apply group discounts.
18  * Apply special offer discounts.
19  * Apply premium feature surcharges.
20  * Validate selected plans and features.
21  * Confirm or cancel the membership process.
22  * Return the final cost as a positive integer when the plan is valid and confirmed.
23  * Return `-1` when the input is invalid or the plan is canceled.
```

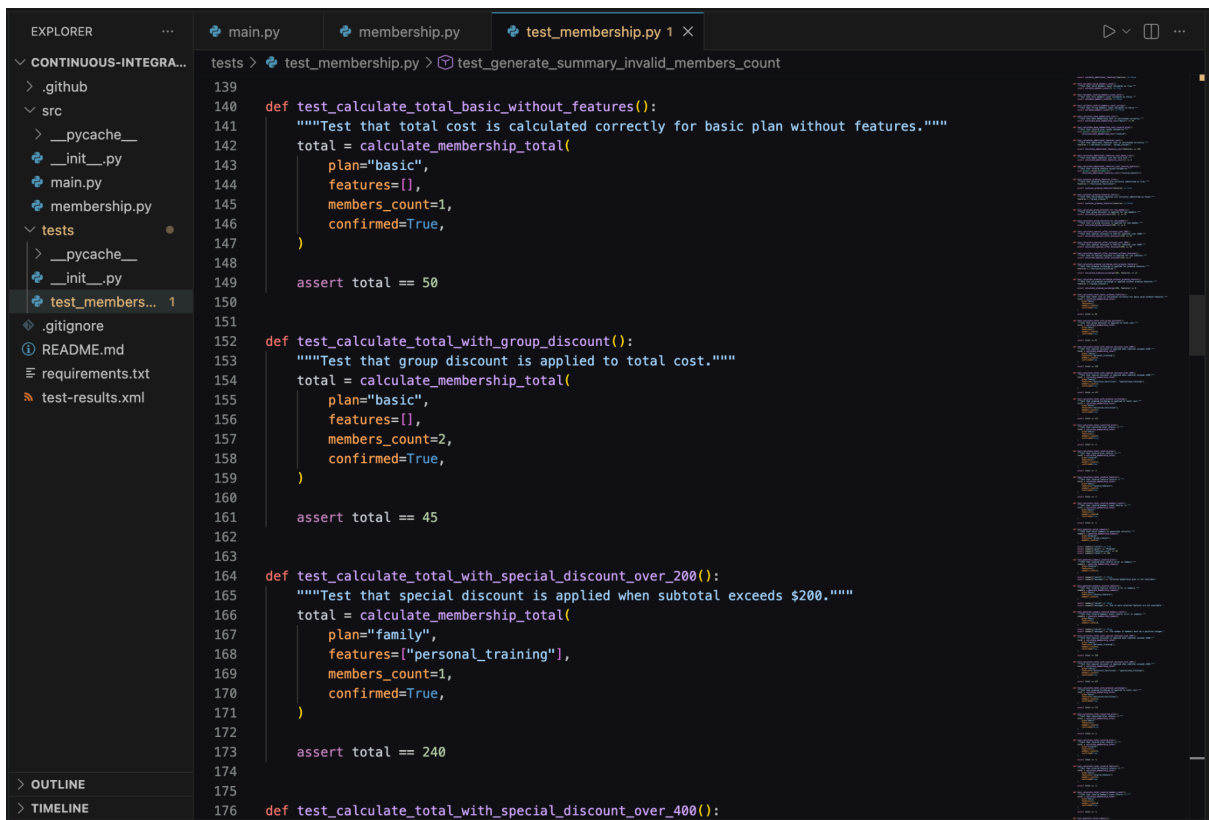
test_membership.py - Dhamar Patiño:



This screenshot shows the Visual Studio Code editor with the file `test_membership.py` open. The interface includes a 'CONTROL DE CÓDIGO FUENTE' sidebar on the left with a 'CAMBIOS' section showing a confirmation message and a 'GRAPH' section displaying a file tree for a pull request. The main editor area shows the following Python code:

```
tests > test_membership.py > test_calculate_total_basic_without_features
1
2 import pytest
3
4 from src.membership import (
5     validate_membership_plan,
6     validate_additional_features,
7     validate_members_count,
8     calculate_base_membership_cost,
9     calculate_additional_features_cost,
10    contains_premium_features,
11    calculate_group_discount,
12    calculate_special_offer_discount,
13    calculate_premium_surcharge,
14    calculate_membership_total,
15    generate_membership_summary,
16 )
17
18
19 def test_validate_existing_membership_plan():
20     """Test that existing membership plan validates as True."""
21     assert validate_membership_plan("basic") is True
22
23
24 def test_validate_invalid_membership_plan():
25     """Test that invalid membership plan validates as False."""
26     assert validate_membership_plan("gold") is False
27
28
```

test_membership.py (Gustavo Guzman)

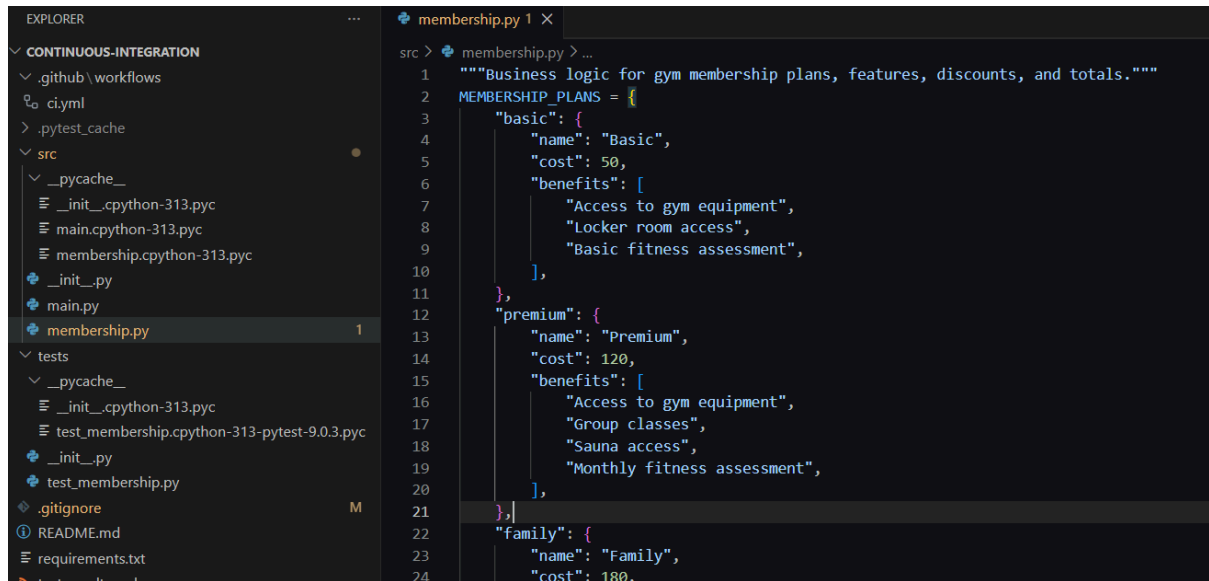


This screenshot shows the Visual Studio Code editor with the file `test_membership.py` open. The interface includes an 'EXPLORER' sidebar on the left showing a project structure with folders like `.github`, `src`, and `tests`. The main editor area shows the following Python code:

```
tests > test_membership.py > test_generate_summary_invalid_members_count
139
140 def test_calculate_total_basic_without_features():
141     """Test that total cost is calculated correctly for basic plan without features."""
142     total = calculate_membership_total(
143         plan="basic",
144         features=[],
145         members_count=1,
146         confirmed=True,
147     )
148     assert total == 50
149
150
151
152 def test_calculate_total_with_group_discount():
153     """Test that group discount is applied to total cost."""
154     total = calculate_membership_total(
155         plan="basic",
156         features=[],
157         members_count=2,
158         confirmed=True,
159     )
160     assert total == 45
161
162
163
164 def test_calculate_total_with_special_discount_over_200():
165     """Test that special discount is applied when subtotal exceeds $200."""
166     total = calculate_membership_total(
167         plan="family",
168         features=["personal_training"],
169         members_count=1,
170         confirmed=True,
171     )
172     assert total == 240
173
174
175
176 def test_calculate_total_with_special_discount_over_400():

```


membership.py (Christopher Villón)



```
1 """Business logic for gym membership plans, features, discounts, and totals."""
2 MEMBERSHIP_PLANS = {
3     "basic": {
4         "name": "Basic",
5         "cost": 50,
6         "benefits": [
7             "Access to gym equipment",
8             "Locker room access",
9             "Basic fitness assessment",
10        ],
11    },
12    "premium": {
13        "name": "Premium",
14        "cost": 120,
15        "benefits": [
16            "Access to gym equipment",
17            "Group classes",
18            "Sauna access",
19            "Monthly fitness assessment",
20        ],
21    },
22    "family": {
23        "name": "Family",
24        "cost": 180,
```

3. Conclusions

The implementation of Continuous Integration through Jenkins and GitHub Actions demonstrated how automation can improve the software development process by ensuring that code changes are continuously validated. By automatically executing unit tests and code quality checks, potential errors can be detected early, reducing development risks and improving the reliability of the application.

Additionally, the development of the Gym Membership Management System provided practical experience in collaborative software development, automated testing, and version control. The workshop highlighted the importance of Continuous Integration in modern software projects, as it helps maintain code quality, facilitates teamwork, and supports the delivery of more stable and maintainable software products.

4. Recommendations

It is recommended to implement Continuous Integration practices from the early stages of software development to detect errors quickly and maintain code quality throughout the project lifecycle.

Development teams should create comprehensive unit tests and keep them updated as new features are added, ensuring that all critical functionalities are properly validated.

Tools such as Jenkins and GitHub Actions should be integrated into collaborative projects to automate testing, code analysis, and other repetitive tasks, reducing manual effort and improving development efficiency.

Developers should follow coding standards and regularly use linting tools to maintain clean, readable, and maintainable code.

Regular monitoring of CI reports and test results is recommended to identify potential issues promptly and prevent defects from reaching production environments.

5. References

- [1] Jenkins, «How to Install Jenkins on Windows,». [En línea]. Available: <https://www.jenkins.io/doc/book/installing/windows/>.
- [2] Jenkins, «Installing Jenkins on Linux,» [En línea]. Available: <https://www.jenkins.io/doc/book/installing/linux/>.